

✓ LR(0) Parser Generation – Initial State

❖ Consider the following grammar $G1$:

$$1: E \rightarrow E + T$$

$$3: T \rightarrow \mathbf{ID}$$

$$2: E \rightarrow T$$

$$4: T \rightarrow (E)$$

❖ For LR parsing, grammars are **augmented** with a . . .

* New start symbol S , and a

* New start production $0: S \rightarrow E \$$

❖ The input should be reduced to E followed by $\$$

* We indicate this by the item: $S \rightarrow \bullet E \$$

❖ The initial state (numbered 0) will have the item: $S \rightarrow \bullet E \$$

❖ An LR parser will start in state 0

❖ State 0 is initially pushed on top of parser stack

Identifying the Initial State

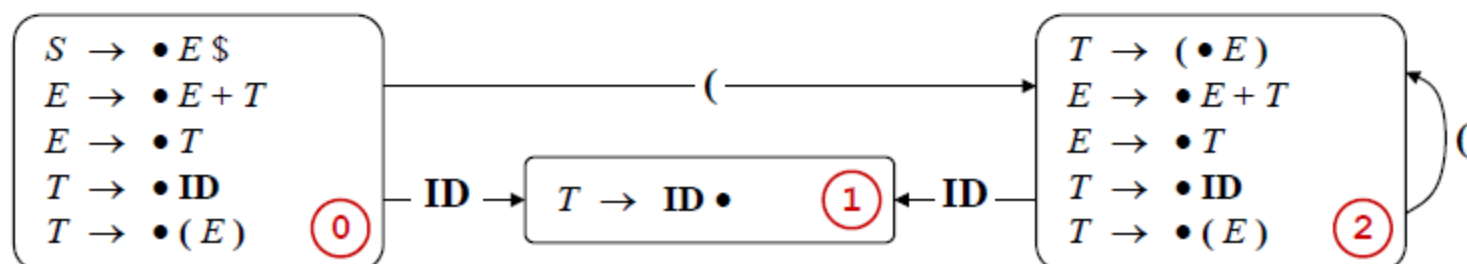
- ❖ Since the dot appears before E , an E is expected
 - * There are two productions of E : $E \rightarrow E + T$ and $E \rightarrow T$
 - * Either $E+T$ or T is expected
 - * The items: $E \rightarrow \bullet E + T$ and $E \rightarrow \bullet T$ are added to the initial state
- ❖ Since T can be expected and there are two productions for T
 - * Either **ID** or (E) can be expected
 - * The items: $T \rightarrow \bullet \text{ID}$ and $T \rightarrow \bullet (E)$ are added to the initial state
- ❖ The initial state (0) is identified by the following set of items

S	$\rightarrow$	$\bullet E \$$
E	$\rightarrow$	$\bullet E + T$
E	$\rightarrow$	$\bullet T$
T	$\rightarrow$	$\bullet \text{ID}$
T	$\rightarrow$	$\bullet (E)$

①

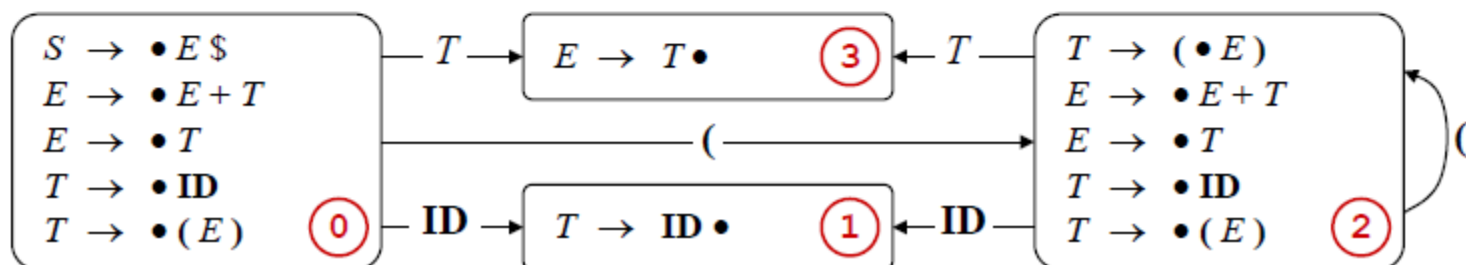
Shift Actions

- ❖ In state 0, we can shift either an **ID** or a left parenthesis
 - * If we shift an **ID**, we shift the dot past the **ID**
 - * We obtain a new item $T \rightarrow \mathbf{ID} \bullet$ and a new state (state 1)
 - * If we shift a left parenthesis, we obtain $T \rightarrow (\bullet E)$
 - * Since the dot appears before E , an E is expected
 - * We add the items $E \rightarrow \bullet E + T$ and $E \rightarrow \bullet T$
 - * Since the dot appears before T , we add $T \rightarrow \bullet \mathbf{ID}$ and $T \rightarrow \bullet (E)$
 - * The new set of items forms a new state (state 2)
- ❖ In State 2, we can also shift an **ID** or a left parenthesis as shown



Reduce and Goto Actions

- ❖ In state 1, the dot appears at the end of item $T \rightarrow \mathbf{ID} \bullet$
 - * This means that **ID** appears on top of stack and can be reduced to T
 - * When $\bullet$ appears at end of an item, the parser can perform a reduce action
- ❖ If **ID** is reduced to T , what is the next state of the parser?
 - * **ID** is popped from the stack; Previous state appears on top of stack
 - * T is pushed on the stack
 - * A new item $E \rightarrow T \bullet$ and a new state (state 3) are obtained
 - * If top of stack is state 0 and we push a T , we go to state 3
 - * Similarly, if top of stack is state 2 and we push a T , we go also to state 3



—

LR(0) Parsing Table

- ❖ The LR(0) parsing table is obtained from the LR(0) state diagram
- ❖ The rows of the parsing table correspond to the LR(0) states
- ❖ The columns correspond to tokens and non-terminals
- ❖ For each state transition $i \rightarrow j$ caused by a token $x \dots$
 - * Put **Shift** j at position $[i, x]$ of the table
- ❖ For each transition $i \rightarrow j$ caused by a nonterminal $A \dots$
 - * Put **Goto** j at position $[i, A]$ of the table
- ❖ For each state containing an item $A \rightarrow \alpha \bullet$ of rule $n \dots$
 - * Put **Reduce** n at position $[i, y]$ for every token y
- ❖ For each transition $i \rightarrow \text{Accept} \dots$
 - * Put **Accept** at position $[i, \$]$ of the table

❖ The LR(0) table of grammar $G1$ is shown below

- * For a shift, the token to be shifted determines the next state
- * For a reduce, the state on top of stack specifies the production to be used

State	Action					Goto	
	+	ID	(	)	\$	E	T
0		S1	S2			G4	G3
1	R3	R3	R3	R3	R3		
2		S1	S2			G6	G3
3	R2	R2	R2	R2	R2		
4	S5				A		
5		S1	S2				G7
6	S5			S8			
7	R1	R1	R1	R1	R1		
8	R4	R4	R4	R4	R4		

Entries are labeled with ...

S_n : Shift token and goto state n
(call scanner for next token)

R_n : Reduce using production n

G_n : Goto state n (after reduce)

A: Accept parse
(terminate successfully)

blank : Syntax error

LR(0) Grammars

- ❖ The shift-reduce conflict in state 0 indicates that G_2 is not LR(0)
- ❖ A grammar is LR(0) if and only if each state is either ...
 - * A **shift state**, containing only shift items
 - * A **reduce state**, containing only a single reduce item
- ❖ If a state contains $A \rightarrow \alpha \bullet x \gamma$ then it cannot contain $B \rightarrow \beta \bullet$
 - * Otherwise, parser can shift x and reduce $B \rightarrow \beta \bullet$ (**shift-reduce conflict**)
- ❖ If a state contains $A \rightarrow \alpha \bullet$ then it cannot contain $B \rightarrow \beta \bullet$
 - * Otherwise, parser can reduce $A \rightarrow \alpha \bullet$ and $B \rightarrow \beta \bullet$ (**reduce-reduce conflict**)
- ❖ LR(0) lacks the power to parse programming language grammars
 - * Because they do not use the lookahead token in making parsing decisions